

Flows Visualization — CDMX

Adaptación de `flows_viz.ipynb` (Mauro et al., 2025) para Ciudad de México.

Genera la **Fig. 4a/4b** del paper: flujos de agentes L (salida) y M+H (entrada) en la celda que gentrificó.

Cambios respecto al original:

- Grid 5x5 (no 7x7)
- `draw_graph` usa `width=height=5`
- Rutas incluyen `mode='improve'` y `starting_deployment='centre_segr-big'`
- `row_of_interest` : celda de la grilla 5x5 (ej. `'(2, 3)'`), ajustar según tu réplica
- `split_x` hardcodedo en el original → aquí parametrizado como `h+1`

```
In [1]: from ABM_MeanField_Cells.utils import *
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from utils_gent_measure import *
import networkx as nx
import matplotlib.gridspec as gridspec
import math
import os

np.set_printoptions(linewidth=400)
%matplotlib inline

pd.options.mode.chained_assignment = None
sns.set()
sns.set(rc={'figure.figsize': (16, 9)})
sns.set_style("whitegrid")
sns.set_context("talk")

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
warnings.simplefilter(action='ignore', category=pd.errors.PerformanceWarning)
```

```
In [5]: # — CDMX: grid 5x5, mode='improve', deployment='centre_segr-big' —————
# CAMBIO: 'centre_segr' → 'centre_segr-big', añadir 'improve' a la ruta
mode = "improve"
starting_deployment = "centre_segr-big"
num_agents = 2**12

directory = f"out/batch_results/{mode}/{starting_deployment}/{num_agents}"
intermediate_dir = os.path.join(directory, "intermediate")
```

```
exp_directory = os.path.join(directory, "exps")

out_plot_directory = os.path.join("out/plots", starting_deployment)
os.makedirs(f"{out_plot_directory}/{num_agents}", exist_ok=True)

print("intermediate:", intermediate_dir)
print("exps:         ", exp_directory)
```

```
intermediate: out/batch_results/improve/centre_segr-big/4096agents/intermediate
exps:         out/batch_results/improve/centre_segr-big/4096agents/exps
```

```
In [6]: # CAMBIO: width=height=5 (era 9 para 'big' o 7 para el default)
width = height = 5

import colorcet as cc
palette = sns.color_palette(cc.glasbey, n_colors=width * height) # 25 color
```

1. Selección de réplica

Elige `p_g` y `rep`. Para encontrar una celda interesante:

- Corre primero el bloque de `cells2times` y busca celdas con `time_gamma > 0`
- Prueba `p_g=0.1` o `p_g=0.15` que tienen mayor gentrificación en CDMX

```
In [7]: # — Parámetros de réplica – ajustar hasta encontrar celda con G_net > 0 —
p_g = 0.1 # probar: 0.01, 0.05, 0.1, 0.15
h = 20
delta = 15
rep = 0 # probar distintas réplicas (0..149)

p_g_str = f"pg_{p_g}_"
h_str = f"h_{h}_"
rep_str = f"rep_{rep}_"
delta_str = f"delta_{delta}_"

chi_hat_filename = f"{p_g_str}{h_str}{rep_str}{delta_str}results_chi_hat."
gamma_filename = f"{p_g_str}{h_str}{rep_str}{delta_str}results_net_avg."
chi_filename = f"{p_g_str}{h_str}{rep_str}{delta_str}results_chi.csv"
exec_model_filename = f"{p_g_str}{h_str}{rep_str}results_model.csv"
exec_agents_filename = f"{p_g_str}{h_str}{rep_str}results_agents.csv"

df_chi_hat = pd.read_csv(os.path.join(intermediate_dir, chi_hat_filename))
df_gamma = pd.read_csv(os.path.join(intermediate_dir, gamma_filename))
df_chi = pd.read_csv(os.path.join(intermediate_dir, chi_filename))
df_exec_model = pd.read_csv(os.path.join(exp_directory, exec_model_filename))

print(f"Celdas totales en G_net: {len(df_gamma.columns)}")
```

Celdas totales en G_net: 25

```
In [9]: # Construir tabla de tiempos de primer evento por celda
cells2times = {}
for cell in df_chi_hat.columns:
```

```

    peaks = find_shifts(df_chi_hat[cell].reset_index(drop=True))
    cells2times[cell] = 0 if len(peaks) == 0 else peaks[0]
cells2times = {k: 0 if np.isnan(v) else v for k, v in cells2times.items()}

dict_times_gamma = {}
for cell in df_gamma.columns:
    peaks_gamma = find_peaks_custom(df_gamma[cell].reset_index(drop=True))
    dict_times_gamma[cell] = 0 if len(peaks_gamma) == 0 else peaks_gamma[0]

df_times_gamma = pd.DataFrame.from_dict(dict_times_gamma, orient="index", columns=df_gamma.columns)
df_times_cells = pd.DataFrame.from_dict(cells2times, orient="index", columns=df_gamma.columns)
df_times = pd.concat([df_times_gamma, df_times_cells], axis=1).fillna(0)

def min_column_with_special_handling(row):
    # FIX pandas >= 2.0: usar .iloc en lugar de row[0], row[1]
    v0 = row.iloc[0]
    v1 = row.iloc[1]
    if v0 == 0 and v1 != 0: return "FN"
    elif v1 == 0 and v0 != 0: return "FP"
    elif v0 == 0 and v1 == 0: return "TN"
    else:
        min_val = row.iloc[:2].min()
        return row.iloc[:2][row.iloc[:2] == min_val].index.tolist()[0]

df_times["min"] = df_times.apply(min_column_with_special_handling, axis=1)
print(df_times)
print("\nCeldas con G_net > 0:", (df_times['time_gamma'] > 0).sum())
df_times["min"] = df_times.apply(min_column_with_special_handling, axis=1)
print(df_times)
print("\nCeldas con G_net > 0:", (df_times['time_gamma'] > 0).sum())

```

Task was destroyed but it is pending!

```

task: <Task pending name='Task-95' coro=<_async_in_context.<locals>.run_in_context() done, defined at /Users/mateos_kvn/Gentrificacion_CDMX/Gentrificacion/.venv/lib/python3.11/site-packages/ipykernel/utils.py:57> wait_for=<Task pending name='Task-96' coro=<Kernel.shell_main() running at /Users/mateos_kvn/Gentrificacion_CDMX/Gentrificacion/.venv/lib/python3.11/site-packages/ipykernel/kernelbase.py:597> cb=[Task.task_wakeup()]> cb=[ZMQStream._run_callback.<locals>._log_error() at /Users/mateos_kvn/Gentrificacion_CDMX/Gentrificacion/.venv/lib/python3.11/site-packages/zmq/eventloop/zmqstream.py:563]> /Users/mateos_kvn/Gentrificacion_CDMX/Gentrificacion/.venv/lib/python3.11/site-packages/pandas/core/internals/blocks.py:650: RuntimeWarning: coroutine 'Kernel.shell_main' was never awaited
    return type(self)(values, placement=self._mgr_locs, ndim=self.ndim, refs=refs)
RuntimeWarning: Enable tracemalloc to get the object allocation traceback
Task was destroyed but it is pending!
task: <Task pending name='Task-96' coro=<Kernel.shell_main() running at /Users/mateos_kvn/Gentrificacion_CDMX/Gentrificacion/.venv/lib/python3.11/site-packages/ipykernel/kernelbase.py:597> cb=[Task.task_wakeup()]>

```

	time_gamma	time_chi_hat	min
(0, 0)	21	0	FP
(0, 1)	99	106	time_gamma
(0, 2)	0	0	TN
(0, 3)	65	94	time_gamma
(0, 4)	51	60	time_gamma
(1, 0)	0	0	TN
(1, 1)	0	0	TN
(1, 2)	0	0	TN
(1, 3)	0	0	TN
(1, 4)	0	0	TN
(2, 0)	0	0	TN
(2, 1)	60	70	time_gamma
(2, 2)	0	0	TN
(2, 3)	0	0	TN
(2, 4)	23	0	FP
(3, 0)	27	31	time_gamma
(3, 1)	125	0	FP
(3, 2)	0	0	TN
(3, 3)	103	108	time_gamma
(3, 4)	23	0	FP
(4, 0)	26	34	time_gamma
(4, 1)	30	39	time_gamma
(4, 2)	40	0	FP
(4, 3)	0	0	TN
(4, 4)	107	113	time_gamma

Celdas con $G_{net} > 0$: 14

	time_gamma	time_chi_hat	min
(0, 0)	21	0	FP
(0, 1)	99	106	time_gamma
(0, 2)	0	0	TN
(0, 3)	65	94	time_gamma
(0, 4)	51	60	time_gamma
(1, 0)	0	0	TN
(1, 1)	0	0	TN
(1, 2)	0	0	TN
(1, 3)	0	0	TN
(1, 4)	0	0	TN
(2, 0)	0	0	TN
(2, 1)	60	70	time_gamma
(2, 2)	0	0	TN
(2, 3)	0	0	TN
(2, 4)	23	0	FP
(3, 0)	27	31	time_gamma
(3, 1)	125	0	FP
(3, 2)	0	0	TN
(3, 3)	103	108	time_gamma
(3, 4)	23	0	FP
(4, 0)	26	34	time_gamma
(4, 1)	30	39	time_gamma
(4, 2)	40	0	FP
(4, 3)	0	0	TN
(4, 4)	107	113	time_gamma

Celdas con $G_{net} > 0$: 14

```
In [10]: # — Seleccionar celda de interés —
# CAMBIO: era '(2, 4)' para grid 7x7. Para 5x5 celdas van de (0,0) a (4,4).
# Elige una celda de df_times donde time_gamma > 0
# Ejemplo: buscar automáticamente la primera celda gentrificada
gent_cells = df_times[df_times['time_gamma'] > 0].index.tolist()
if gent_cells:
    row_of_interest = gent_cells[0]
    print(f"Celda seleccionada automáticamente: {row_of_interest}")
    print(f"Otras celdas gentrificadas: {gent_cells}")
else:
    print("⚠ No hay celdas con G_net > 0 en esta réplica. Cambia p_g o rep.")
    row_of_interest = df_gamma.columns[0] # fallback
```

Celda seleccionada automáticamente: (0, 0)

Otras celdas gentrificadas: ['(0, 0)', '(0, 1)', '(0, 3)', '(0, 4)', '(2, 1)', '(2, 4)', '(3, 0)', '(3, 1)', '(3, 3)', '(3, 4)', '(4, 0)', '(4, 1)', '(4, 2)', '(4, 4)']

```
In [14]: # get_gini_richness_df → OMITIDO: model_cdmx no recolecta Gini

df = read_df_agents(os.path.join(exp_directory, exec_agents_filename))

# Definir all_sources PRIMERO
mass = range(df["pos"].max()[0] + 1)
num_steps = max(df["Step"])
all_sources = [(o, d) for o in mass for d in mass]

# Ahora sí llamar get_median_richness_df con all_sources
df_cell_median = get_median_richness_df(df_exec_model, all_sources)

df_edges = df[df["source"] != df["pos"]].reset_index(drop=True)
df_edges_A, df_edges_B, df_edges_C = get_df_edges(df_edges)

df_flow_A = get_df_flows(df_edges_A)
df_flow_B = get_df_flows(df_edges_B)
df_flow_C = get_df_flows(df_edges_C)

# Primer pico de G_net en la celda seleccionada
if row_of_interest in df_gamma.columns:
    first_peak = find_peaks_custom(
        df_gamma[row_of_interest].reset_index(drop=True))[0]
    print(f"Primer pico G_net en celda {row_of_interest}: paso {first_peak}")
else:
    print(f"⚠ Celda {row_of_interest} no encontrada en df_gamma.columns")
    print("Columnas disponibles:", list(df_gamma.columns[:5]))
```

Primer pico G_net en celda (0, 0): paso 21

2. Cherry-pick: G_bin vs G_net en la celda seleccionada

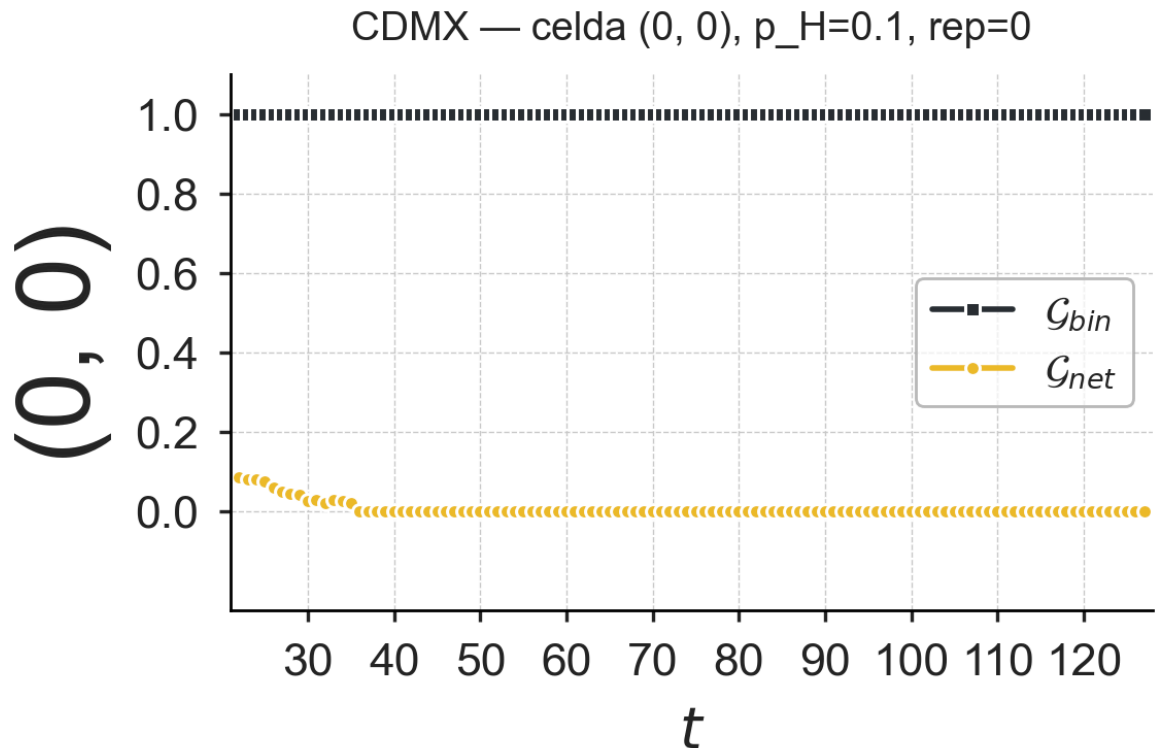
Figura equivalente a Fig. 4a del paper.

```

In [15]: def plot_cherry_picking(ax, df_chi, df_chi_hat, df_gamma, row_of_interest, l
sns.lineplot(ax=ax, data=df_chi_hat[row_of_interest].reset_index(drop=True),
              linewidth=linewidth, dashes=False, label=r"$\mathcal{G}_{bi}$",
              marker="s", color='#2A2E34', markersize=markersize, markere
sns.lineplot(ax=ax, data=df_gamma[row_of_interest].reset_index(drop=True),
              linewidth=linewidth, dashes=False, label=r"$\mathcal{G}_{ne}$",
              marker="o", color='#eeba2c', markersize=markersize, markere
sns.despine(ax=ax)
ax.grid(linestyle="--", linewidth=0.25)
ax.set_xticks(np.arange(0, len(df_chi_hat[row_of_interest]), 10))
y_ticks = np.arange(0, 1.1, 0.2)
ax.set_yticks(y_ticks)
ax.set_yticklabels([f"{y:.1f}" for y in y_ticks], fontsize=12)
ax.spines['bottom'].set_color('black')
ax.spines['left'].set_color('black')
ax.spines['bottom'].set_linewidth(0.5)
ax.spines['left'].set_linewidth(0.5)
ax.tick_params(axis='both', bottom=True, left=True, labelsz=8, length=
leg = ax.legend(title='', fontsize=7, frameon=True)
leg.get_frame().set_edgecolor('darkgrey')
leg.get_frame().set_linewidth(.5)
# Mostrar desde ~h hasta el final
ax.set_xlim(h + 1, len(df_gamma))
ax.set_ylim(-0.25, 1.1)
ax.set_xlabel(r"$t$", fontsize=10)
plt.tight_layout()
return ax

fig, ax = plt.subplots(figsize=(3.3, 2.2), dpi=400)
ax = plot_cherry_picking(ax, df_chi, df_chi_hat, df_gamma, row_of_interest)
ax.set_title(f"CDMX - celda {row_of_interest}, p_H={p_g}, rep={rep}", fontsi
plt.savefig(f"{out_plot_directory}/{num_agents}/cdmx_cherry_pg{p_g}_rep{rep}
plt.show()

```



3. Grafos de flujo: salida L y entrada M+H

Figura equivalente a Fig. 4b del paper. Muestra los flujos de agentes L (rojo) que salen y M+H (azul-verde) que entran en la celda focal.

```
In [16]: def add_dataframes(df1, df2):
    merged = pd.merge(df1, df2, on=['source', 'pos'], how='outer', suffixes=
    merged['flow_first'] = merged['flow_first'].fillna(0)
    merged['flow_second'] = merged['flow_second'].fillna(0)
    merged['flow'] = merged['flow_first'] + merged['flow_second']
    return merged[['source', 'pos', 'flow']]

def get_graph_data(cell, step, delta, df_flow_C, df_flow_B, df_flow_A, df_ce
    df_med_step = df_cell_median[df_cell_median.index == step - delta]
    cell_richness = dict(df_med_step.iloc[0])
    cell_richness = {k: int(v) for k, v in cell_richness.items() if not math
    cell_richness_div = {k: v / 100 for k, v in cell_richness.items()}

    def slice_flows(df_flow, cell, step, delta):
        df_step = df_flow[(df_flow["Step"] >= step - delta) & (df_flow["Step
        out = df_step[df_step["source"] == cell].groupby(["source", "pos"]).
        inn = df_step[df_step["pos"] == cell].groupby(["source", "pos"]).sum
        return out, inn

    Cout, _Cin = slice_flows(df_flow_C, cell, step, delta)
    Aout, Ain = slice_flows(df_flow_A, cell, step, delta)
    Bout, Bin = slice_flows(df_flow_B, cell, step, delta)
    ABin = add_dataframes(Ain, Bin)
```

```

return {"weighted_edgelist_Cout_step": Cout,
        "weighted_edgelist_ABin_step": ABin,
        "cell_richnessess_beginningstep_div": cell_richness_div}

```

```

In [17]: def draw_graph(ax, weighted_edgelist, title, classe="C", focus="max", cell_r
# CAMBIO: width=height=5 (era 7 hardcodeado en el original)
W = H = 5
focus_node = eval(row_of_interest) if focus == "max" else None

ax.grid(alpha=0.5, linestyle="--", linewidth=0.3)
ax.set_frame_on(False)
ax.set_xlim(-0.5, W - 1 + 0.5)
ax.set_ylim(-0.5, H - 1 + 0.5)
ax.set_aspect('equal')
ax.set_xticks(range(W))
ax.set_yticks(range(H))
ax.set_xticklabels([])
ax.set_yticklabels([])

G = nx.DiGraph()
for x in range(W):
    for y in range(H):
        G.add_node((x, y))

for entry in weighted_edgelist:
    G.add_edge(entry['source'], entry['pos'], weight=entry['flow'])

positions = {node: (node[1], H - 1 - node[0]) for node in G.nodes()}

for node in G.nodes():
    color = 'black' if node == focus_node else 'grey'
    size = cell_richnessess_beginningstep.get(node, 0) / 20 if cell_ric
    nx.draw_networkx_nodes(G, pos=positions, nodelist=[node],
                           node_size=size, node_color=color, ax=ax, line

for edge in G.edges(data=True):
    color = "#5fbdb5" if classe == "B" else "#f1938e"
    nx.draw_networkx_edges(G, pos=positions, edgelist=[(edge[0], edge[1]
                                                              width=edge[2]['weight'] / 5, edge_color=color
                                                              arrows=True, arrowstyle='->', arrowsize=6,
                                                              connectionstyle='arc3,rad=0.35',
                                                              min_source_margin=0, min_target_margin=0, ax=

ax.set_title(title, fontsize=6, loc='left', fontweight='bold')

```

```

In [18]: # — Grafo EN el momento del primer pico —————
cell     = eval(row_of_interest)
step     = first_peak
results  = get_graph_data(cell, step, delta, df_flow_C, df_flow_B, df_flow_A,

fig = plt.figure(figsize=(6.6, 3.3), dpi=400)
gs  = gridspec.GridSpec(1, 2, width_ratios=[1, 1], wspace=0.15)
ax1 = fig.add_subplot(gs[0])
ax2 = fig.add_subplot(gs[1])

```



```

draw_graph(ax1, results["weighted_edgelist_Cout_step"].to_dict('records'),
            f"Salida L (t={step})", classe="C", focus="max",
            cell_richnessess_beginningstep=results["cell_richnessess_beginningstep"],
draw_graph(ax2, results["weighted_edgelist_ABin_step"].to_dict('records'),
            f"Entrada M+H (t={step})", classe="B", focus="max",
            cell_richnessess_beginningstep=results["cell_richnessess_beginningstep"],

plt.suptitle(f"CDMX — celda {row_of_interest}, p_H={p_g}, rep={rep} — durante",
plt.tight_layout()
plt.savefig(f"{out_plot_directory}/{num_agents}/cdmx_flows_during_pg{p_g}_rep{rep}.png")
plt.show()

```

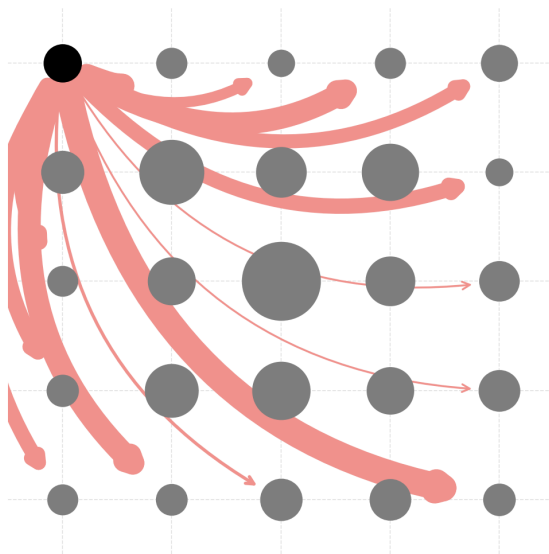
```

/var/folders/rw/j2_s688s5x9f9k_xg2xhfhq980000gn/T/ipykernel_2732/1885106240.py:19: UserWarning: This figure includes Axes that are not compatible with tight_layout, so results might be incorrect.
plt.tight_layout()

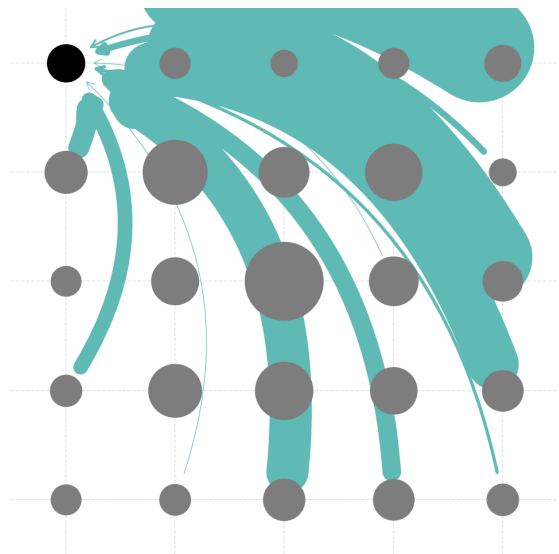
```

CDMX — celda (0, 0), p_H=0.1, rep=0 — durante

Salida L (t=21)



Entrada M+H (t=21)



```

In [19]: # — Grafo POST pico (first_peak + 8 pasos) —————
step      = first_peak + 8
results = get_graph_data(cell, step, delta, df_flow_C, df_flow_B, df_flow_A,

fig = plt.figure(figsize=(6.6, 3.3), dpi=400)
gs   = gridspec.GridSpec(1, 2, width_ratios=[1, 1], wspace=0.15)
ax1  = fig.add_subplot(gs[0])
ax2  = fig.add_subplot(gs[1])

draw_graph(ax1, results["weighted_edgelist_Cout_step"].to_dict('records'),
            f"Salida L (t={step})", classe="C", focus="max",
            cell_richnessess_beginningstep=results["cell_richnessess_beginningstep"],
draw_graph(ax2, results["weighted_edgelist_ABin_step"].to_dict('records'),
            f"Entrada M+H (t={step})", classe="B", focus="max",
            cell_richnessess_beginningstep=results["cell_richnessess_beginningstep"],

plt.suptitle(f"CDMX — celda {row_of_interest}, p_H={p_g}, rep={rep} — post",
plt.tight_layout()

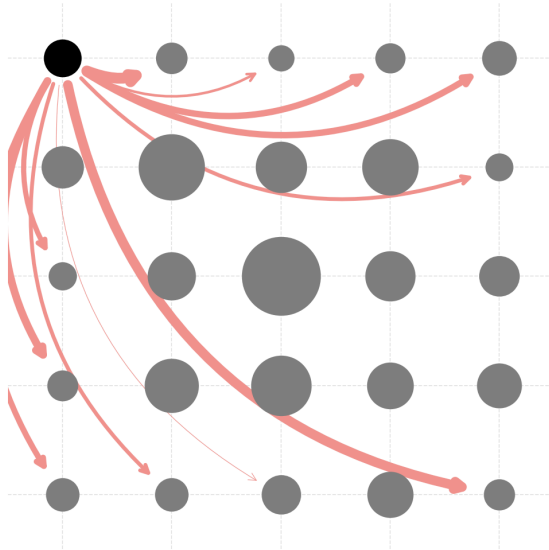
```

```
plt.savefig(f"{out_plot_directory}/{num_agents}/cdmx_flows_post_pg{p_g}_rep{p_g}_t{t}.png")
plt.show()
print(f"first_peak={first_peak}, step_post={step}")
```

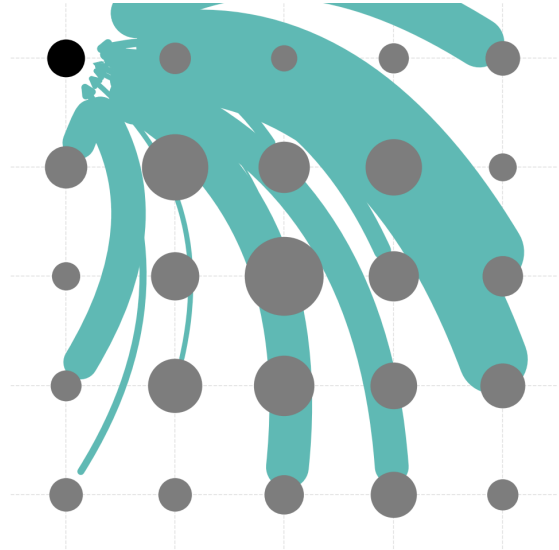
```
/var/folders/rw/j2_s688s5x9f9k_xg2xhfhq980000gn/T/ipykernel_2732/965256688.py:18: UserWarning: This figure includes Axes that are not compatible with tight_layout, so results might be incorrect.
  plt.tight_layout()
```

CDMX — celda (0, 0), p_H=0.1, rep=0 — post

Salida L (t=29)



Entrada M+H (t=29)



first_peak=21, step_post=29

In []: